



Implementation Plan

Architecture · RAG · MCP · Options

RGAA KNOWLEDGE HUB

Février 2026 — Confidentiel

Accessibilité Numérique · Référentiel RGAA 4.1
ChromaDB · Embeddings · MCP Protocol · React



Table des Matières

RGAA Knowledge Hub — Implementation Plan

Contexte et base existante

Architecture cible

Structure de fichiers cible

Proposed Changes

Couche 0 — Refactoring préalable (sans breaking change)

Couche 1 — Système de configuration persistée (Settings)

Couche 2 — Base vectorielle et embeddings

Couche 3 — Sources d'apprentissage

Couche 4 — RAG Engine

Couche 5 — Hook post-import (non-bloquant)

Couche 6 — Nouveaux outils MCP

Couche 7 — Menu Options (UI React)

Ordre d'implémentation

Plan de vérification

Tests automatisés

Test d'intégration MCP

Test de résilience ChromaDB

Test end-to-end Hub

RGAA Knowledge Hub — Implementation Plan

Objectif : Transformer le projet RGAA Extractor v1.0 en un **Hub intelligent** — un cerveau centralisé qui ingère les rapports d'audit, apprend de ses données, et expose cette connaissance à n'importe quel client via le protocole MCP.

Hors scope : comparaison de rapports (`compare_reports` , `get_conformity_rate`).

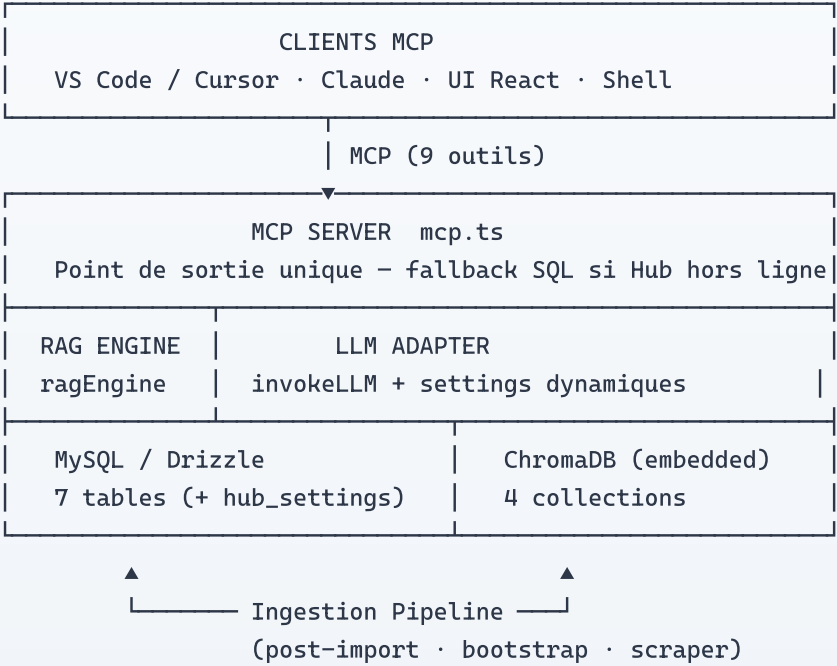
Contexte et base existante

Le projet dispose déjà d'une base solide qui sera **conservée intégralement** :

COMPOSANT	ÉTAT	NOTES
MySQL + Drizzle ORM	✓ Conservé	6 tables en production
Parser DOCX (mammoth.js)	✓ Conservé	Extraction structurée éprouvée
MCP Server	✓ Étendu	5 outils → 9 outils
<code>invokeLLM</code> + Forge API	✓ Wrappé	Nouveau <code>llmAdapter.ts</code> dynamique
Déduplication SHA-256 + Levenshtein	✓ Conservé	Complété par similarité vectorielle
<code>GeneralizationPipeline</code>	✓ Conservé	Nettoyage des templates inchangé

Ce qui est absent et constitue le Hub : ChromaDB, RAG Engine, embeddings, settings persistés.

Architecture cible



Règle d'or — dépendances à sens unique :

repositories/ ← hub/ ← routers/ ← mcp.ts

Aucun repository n'importe depuis **hub/** . Aucun cycle possible.

Structure de fichiers cible

```

server/
├─ _core/                # Framework (inchangé)
│   └─ env.ts            # [MODIFY] + vars Hub
├─ repositories/         # Accès données (inchangé + 1 nouveau)
│   └─ settingsRepository.ts [NEW]
├─ hub/                  # [NEW] Cerveau – couche strictement isolée
│   ├── index.ts         # Exports publics du Hub
│   ├── settingsService.ts # getSetting/setSetting + cache mémoire
│   ├── chromaClient.ts  # Connexion ChromaDB PersistentClient
│   ├── embeddings.ts    # embed() abstrait : OpenAI | Ollama
│   ├── llmAdapter.ts    # invokeHubLLM() – modèle piloté par settings
│   ├── ragEngine.ts     # Moteur RAG central
│   ├── ingestionPipeline.ts # indexReportFindings + bootstrapFromExisting
│   ├── knowledgeScraper.ts # Scraping RGAA / WCAG / WAI-ARIA / AcceDe / MDN
│   └─ data/
│       └─ rgaa-4.1.2.json # Fallback local garanti si scraper indisponible
├─ routers/              # [REFACTOR] Éclatement de routers.ts (468L)
│   ├── index.ts         # Composition finale de appRouter
│   ├── audit.ts         # Migré depuis routers.ts
│   ├── criteria.ts      # Migré depuis routers.ts
│   ├── findingTemplates.ts # Migré depuis routers.ts
│   ├── hub.ts           [NEW] # ask, analyze, suggest, validate
│   └─ settings.ts       [NEW] # CRUD hub_settings + réindexation
├─ utils/                # Inchangé
├─ db.ts                 # Barrel (+ export settingsRepository)
└─ mcp.ts                # Étendu : 5 → 9 outils

```

Proposed Changes

Couche 0 — Refactoring préalable (sans breaking change)

[!IMPORTANT] Ce refactoring est un prérequis bloquant. **routers.ts** (468 lignes, monolithique) doit être éclaté avant d'ajouter le moindre code Hub. Le type **AppRouter** exporté reste identique — zéro impact sur le client React ou le MCP.

[MODIFY] **server/routers.ts** → **server/routers/index.ts**

- Déplacer chaque sous-router dans son propre fichier (**audit.ts** , **criteria.ts** , **findingTemplates.ts**)

- `index.ts` compose le router final :
`router({ system, auth, audit, criteria, findingTemplates })`

Couche 1 — Système de configuration persistée (Settings)

Le Hub est entièrement piloté par des paramètres en base de données. Les variables `.env` servent uniquement de valeurs initiales et de fallback au démarrage.

[MODIFY] `drizzle/schema.ts` — Table `hub_settings`

```
hub_settings {
  key          VARCHAR(100) PRIMARY KEY    -- ex: "llm.model"
  value        TEXT                        -- ex: "openai/gpt-oss-120b:free"
  type         ENUM(string, number, boolean, password)
  category     VARCHAR(50)                -- llm | embed | chroma | rag | sources | mcp
| ui
  description  TEXT
  updatedAt   TIMESTAMP
}
```

[NEW] `server/repositories/settingsRepository.ts`

- `getSetting(key)`, `setSetting(key, value)`, `getAllSettings()`,
`getSettingsByCategory(category)`
- Seed initial des valeurs par défaut à la première migration

[NEW] `server/hub/settingsService.ts`

- Cache mémoire (`Map<string, string>`) — évite N requêtes MySQL par appel LLM
- `warmupCache()` — chargement au démarrage, **fallback silencieux sur ENV si MySQL indisponible**
- `getSetting(key): Promise<string | null>` — toujours disponible, jamais crashant
- `setSetting(key, value)` — invalide le cache localement après écriture

[MODIFY] `server/_core/env.ts`

```
OPENROUTER_API_KEY // Clé OpenRouter
OPENAI_API_KEY     // Clé OpenAI (embeddings)
OLLAMA_HOST        // URL Ollama (défaut: http://localhost:11434)
CHROMA_PATH        // Chemin local ChromaDB (défaut: ./data/chromadb)
```

Couche 2 — Base vectorielle et embeddings

[NEW] `server/hub/chromaClient.ts`

[!NOTE] ChromaDB est utilisé en mode **PersistentClient** (embedded) — aucun processus Python, aucun port externe. Les données persistent dans `./data/chromadb`.

- `isAvailable(): Promise<boolean>` — health check systématique avant chaque appel
- `getOrCreateCollection(name, metadata)` — gestion des 4 collections
- Exporté en singleton (une seule connexion partagée)

4 collections ChromaDB :

COLLECTION	CONTENU	NATURE
<code>rgaa_referential</code>	106 critères + 13 thématiques RGAA 4.1 + WCAG 2.2	Vérité immuable officielle
<code>rgaa_findings</code>	Constats validés des rapports importés	Expertise capitalisée, pondérée
<code>rgaa_expertise</code>	Templates <code>approved</code> + patterns AcceDe Web / MDN	Expertise capitalisée
<code>rgaa_code</code>	Snippets HTML/ARIA before/after par critère (WAI-ARIA APG)	Enrichissement sémantique

Idempotence : tous les documents sont indexés via `.upsert({ ids: [signatureHash] })` — jamais de doublon vectoriel même en cas de re-indexation.

[NEW] `server/hub/embeddings.ts`

Interface abstraite — le Hub ne sait pas quel moteur il utilise.

```
interface EmbeddingProvider {  
  embed(text: string): Promise<number[]>;  
  dimension: number;  
}
```

- **OpenAI** (`text-embedding-3-small` , dim=1536) — si `embed.provider = "openai"`
- **Ollama** (`nomic-embed-text` , dim=768) — si `embed.provider = "ollama"` (défaut)
- Sélection dynamique depuis `settingsService.getSetting("embed.provider")`

[NEW] `server/hub/llmAdapter.ts`

Wrapper au-dessus de `invokeLLM` qui injecte les paramètres dynamiques depuis les settings :

```
export async function invokeHubLLM(params: InvokeParams): Promise<InvokeResult>
```

- Lit `llm.provider` , `llm.model` , `llm.apiKey` , `llm.maxTokens` , `llm.temperature` depuis `settingsService`
- Résout l'URL de base selon le provider (`openrouter.ai` , `api.openai.com` , `ollama` , `forge`)
- Ajoute les headers spécifiques OpenRouter (`HTTP-Referer` , `X-Title`) si nécessaire
- **Ne modifie pas** `invokeLLM` — zéro breaking change

Couche 3 — Sources d'apprentissage

Le Hub apprend de deux types de sources : officielles (indexation initiale) et humaines (flux continu).

[!IMPORTANT] **Règle fondamentale — Qualité > Quantité** : un constat approuvé par un auditeur humain (`confidence=100`) vaut 100 documents extraits automatiquement.

Sources officielles — Ingestion initiale

SOURCE	URL	COLLECTION	FRÉQUENCE
RGAA 4.1.2 — Critères & Glossaire	accessibilite.numerique.gouv.fr	<code>rgaa_referential</code>	One-shot + évolutions
WCAG 2.2 — Techniques	w3.org/WAI/WCAG22/Techniques	<code>rgaa_referential</code>	Évolutions majeures
WAI-ARIA APG — Design Patterns	w3.org/WAI/ARIA/apg/patterns	<code>rgaa_code</code>	Trimestriel
AcceDe Web — Notices	accede-web.com/notices	<code>rgaa_expertise</code>	Mensuel
MDN — Accessibility	developer.mozilla.org/Accessibility	<code>rgaa_expertise</code>	Mensuel

Sources vivantes — Capitalisation continue

SOURCE	MÉCANISME	COLLECTION
Constats validés par les auditeurs	Outil MCP <code>validate_finding</code>	<code>rgaa_findings</code>
Templates <code>approved</code> de la bibliothèque	Auto-sync post-approbation	<code>rgaa_expertise</code>
Constats existants v1.0	Bootstrap one-shot au démarrage	<code>rgaa_findings</code>

[NEW] `server/hub/knowledgeScraper.ts`

Chaque fonction de scraping retourne `{ success, count, errors[] }` — un seuil minimum de documents est validé avant confirmation de l'indexation. Si le seuil n'est pas atteint, l'indexation est annulée et loggée.

Fallback garanti : si le scraping RGAA échoue, `server/hub/data/rgaa-4.1.2.json` est utilisé automatiquement.

Couche 4 — RAG Engine

[NEW] `server/hub/ragEngine.ts`

Flux de traitement d'une question :

```
question entrante
  → embed(question) // vecteur numérique
  → ChromaDB.search(4 collections, topK) // documents similaires
  → scoring(similarité × confiance × occurrenceCount) // pondération
  → construction du prompt enrichi // contexte injecté
  → invokeHubLLM() // réponse LLM
  → retour avec sources citées
```

Fonctions publiques exposées :

FONCTION	DESCRIPTION
<code>askAccessibility(question, context?)</code>	Question libre → réponse RAG + sources
<code>analyzeCode(html, context?)</code>	HTML → non-conformités RGAA détectées + critères
<code>suggestFix(problem, criterionRef?)</code>	Problème → correction concrète + exemple de code
<code>indexDocument(doc, collection)</code>	Indexation d'un document dans ChromaDB

Paramètres RAG configurables depuis les Options :

PARAMÈTRE	DÉFAUT	RÔLE
<code>rag.topK</code>	<code>5</code>	Nb de résultats par collection
<code>rag.minSimilarity</code>	<code>0.65</code>	Seuil de pertinence minimum
<code>rag.weightFindings</code>	<code>1.5</code>	Boost sur les constats humains
<code>rag.weightReferential</code>	<code>1.0</code>	Poids référentiel officiel
<code>rag.weightCode</code>	<code>0.8</code>	Poids snippets de code
<code>rag.citeSources</code>	<code>true</code>	Inclure les sources dans la réponse

[NEW] `server/hub/ingestionPipeline.ts`

- `indexReportFindings(reportId)` — indexe tous les constats d'un rapport (appelé en fire-and-forget post-import)
- `bootstrapFromExistingFindings()` — migration one-shot des constats v1.0 vers ChromaDB

Couche 5 — Hook post-import (non-bloquant)

[MODIFY] `server/routers/audit.ts`

Après `updateAuditReportStatus(reportId, 'completed')`, déclencher l'indexation en tâche de fond :

```
// Pattern fire-and-forget – une erreur ChromaDB ne fait JAMAIS échouer l'import
setImmediate(async () => {
  try {
    await ingestionPipeline.indexReportFindings(reportId);
  } catch (e) {
    console.error("[Hub] Indexation ChromaDB non bloquante:", e);
  }
});
```

L'import DOCX est **totalelement indépendant** de ChromaDB. Le Hub enrichit en arrière-plan.

Couche 6 — Nouveaux outils MCP

[MODIFY] `server/mcp.ts` — 5 outils → 9 outils

4 nouveaux outils :

OUTIL	FAMILLE	ENTRÉE	SORTIE
<code>ask_accessibility</code>	Expertise	<code>question: string, context?: string</code>	Réponse RAG + sources citées
<code>analyze_code</code>	Expertise	<code>html: string, context?: string</code>	Non- conformités + critères + impact
<code>suggest_fix</code>	Expertise	<code>problem: string, criterionRef?: string</code>	Correction + exemple de code
<code>validate_finding</code>	Capitalisation	<code>finding, criterionRef, impact, solution?</code>	Indexation ChromaDB + MySQL

2 outils existants enrichis :

- `search_rgaa_expertise` : fusion résultats SQL + vectoriels
- `propose_deduplication` : Levenshtein existant + similarité vectorielle

Résilience : chaque outil vérifie `chromaClient.isAvailable()` avant tout appel vectoriel. Si ChromaDB est indisponible, la réponse SQL dégradée est retournée avec un flag `"mode": "degraded"` .

1 nouvel outil configuration :

- `get_hub_config` — expose la configuration courante du Hub au client MCP

Couche 7 — Menu Options (UI React)

[NEW] `server/routers/settings.ts`

Router tRPC exposant : `getAll` , `get` , `set` , `triggerReindex(source)` , `getIndexationStatus()`

[NEW] `client/src/pages/Options.tsx`

6 sections en accordéon :

SECTION	PARAMÈTRES CLÉS
 Modèle IA	Provider (OpenRouter/OpenAI/Ollama/Forge), modèle, clé API, température, niveau de détail
 Embeddings	Provider, modèle, URL Ollama, clé OpenAI
 ChromaDB	Chemin local, toggle on/off, statut de connexion live
 RAG Tuning	Top-K, seuil similarité, pondérations par collection, citation des sources
 Sources	Toggle par source + date dernière sync + bouton "Re-indexer maintenant"
 MCP	Toggle outils individuels + générateur <code>mcp.json</code> pour VS Code/Cursor

Ordre d'implémentation

#	ÉTAPE	TYPE	RISQUE SI IGNORÉ
1	Refactoring <code>routers.ts</code> → <code>routers/</code>	Refactor	Dette explosive
2	<code>hub_settings</code> schema + migration Drizzle	BDD	Settings non persistés
3	<code>settingsRepository</code> + <code>settingsService</code>	Service	Race condition démarrage
4	<code>chromaClient</code> (PersistentClient embedded)	Infrastructure	Python requis sans ça
5	<code>embeddings</code> (OpenAI / Ollama abstrait)	Service	Vendor lock-in
6	<code>llmAdapter</code> (wrapper dynamique)	Service	Modèle hardcodé
7	<code>ragEngine</code> + tests unitaires	Core	Hub instable
8	<code>ingestionPipeline</code> + hook fire-and-forget	Pipeline	KB jamais alimentée
9	<code>knowledgeScraper</code> + fallback RGAA local	Pipeline	Sources fragiles
10	Extension <code>mcp.ts</code> (9 outils + fallback SQL)	MCP	MCP inutilisable hors ligne
11	Router <code>settings.ts</code> + <code>Options.tsx</code>	UI	Hub non configurable

Plan de vérification

Tests automatisés

```
pnpm test # Vitest - parser.test.ts + auth.logout.test.ts (existants)
```

Nouveaux tests à créer :

- `server/hub/embeddings.test.ts` — vérifie dimension vecteur, cosine similarity > 0.8 entre phrases similaires, fallback Ollama
- `server/hub/ragEngine.test.ts` — mock ChromaDB + invokeLLM, vérifie construction du contexte enrichi
- `server/hub/settingsService.test.ts` — vérifie le cache, le fallback ENV, l'invalidation

Test d'intégration MCP

```
# Terminal 1 - démarrer le serveur MCP
pnpm mcp

# Terminal 2 - MCP Inspector
npx @modelcontextprotocol/inspector studio "pnpm mcp"
```

Critères de validation :

- Les 9 outils apparaissent dans la liste
- `ask_accessibility({ question: "images sans alternative texte" })` retourne une réponse avec sources
- `ask_accessibility` répond en mode dégradé si ChromaDB est stoppé (fallback SQL visible)

Test de résilience ChromaDB

1. Démarrer le serveur (`pnpm dev`)
2. Stopper ChromaDB (renommer le répertoire `./data/chromadb`)
3. Appeler `ask_accessibility` via MCP Inspector
4. Vérifier : réponse SQL dégradée reçue, serveur toujours opérationnel, flag `"mode": "degraded"` présent

Test end-to-end Hub

1. `pnpm dev`
2. Uploader un rapport DOCX via l'UI

3. Attendre `status = completed`
4. Vérifier dans les logs serveur : `[Hub] Indexation ChromaDB: X constats indexés`
5. Via MCP : `ask_accessibility({ question: "X" })` → réponse cite le rapport importé